

Lecture 3

Separation of Concerns

CS 12 Team 25.2
A.Y. 2025–2026, 2nd Sem
Department of Computer Science
College of Engineering
University of the Philippines Diliman

Sanity Check

- Attendance?
- Audible at the back?
- TV(s) on?
 - (we're working on getting the other TV fixed...)
- **Recording?**
- Important announcements?

Aside: Pylint Strict Mode

```
# pylint: strict
```

```
def my_function(...) -> ...:  
    ...
```

Aside: Lab 1

How was Lab 1?

Now that she's become a student in Millennium Science School, Kei has to take $n + 1$ supplementary tests, where each test is scored an integer between 0 and 100 inclusive.

Kei thought the tests were too easy, so she decided to intentionally get a score of $s_1, s_2, \dots, s_n$ in the first n tests.

Aris is seriously worried that Kei might get a failing average. What is the minimum score that Kei needs to get in the $(n + 1)^{\text{th}}$ test to get an average of **at least** 60?

If it is impossible for Kei to get a passing average, return -1 .

```
assert min_to_pass([50, 70]) == 60
assert min_to_pass([10, 20]) == -1
assert min_to_pass([100, 100]) == 0 # not -1!
```

Aside: Lab 1

```
def min_to_pass(seq: Sequence[int]) -> int:
    return min(
        (score for score in range(100 + 1) \
         if sum(seq) + score >= 60 * (len(seq) + 1)),
        default=-1
    )
```

- Moral: *Always start with brute force!*
 - Very important for CS 32/33

Aside: Index Card

3" × 5" index card should contain the following:

- 1" × 1" picture
- Name
- Student number
- HS attended
- Email address
- Birthday
- Section
- Message to your future self


Hoshino Takanashi
2021-12497
Abydos High School
thoshino@up.edu.ph
January 2, ???
HGI/FVW2
いい夢を見れますように～

Submit this Friday or next Friday

Motivating Example

Consider the following text-based number guessing game:

03/gg_vanilla

Are you satisfied with this?

Modularization

- **Modularization:** Splitting of code into **smaller, reusable** components called **modules**
- **Module:** Single function, group of functions, objects
 - Can help minimize duplication of code

Q1. How *modularized* is the code of the number guessing game?

Q2. How should we modularize *our* code?

Separation of Concerns

- **Separation of concerns:** Modules should be in charge of *nonoverlapping* parts or aspects of solution
- “Gather together things that change for the same reason; separate those that don’t”
- Fewer concerns
 - ⇒ Fewer reasons to change
 - ⇒ Less likely to be changed
 - ⇒ More isolated changes
 - ⇒ More manageable change

Separation of Concerns

- What kinds of concerns does the number guessing game code have?
 - Game logic
 - User input
 - Displaying of output
- We can adjust the *level of granularity* later (if needed)
- To what degree are these concerns *separated* in the code?

Coupling and Dependencies

- **Dependency:** If A depends on B, A cannot exist without B

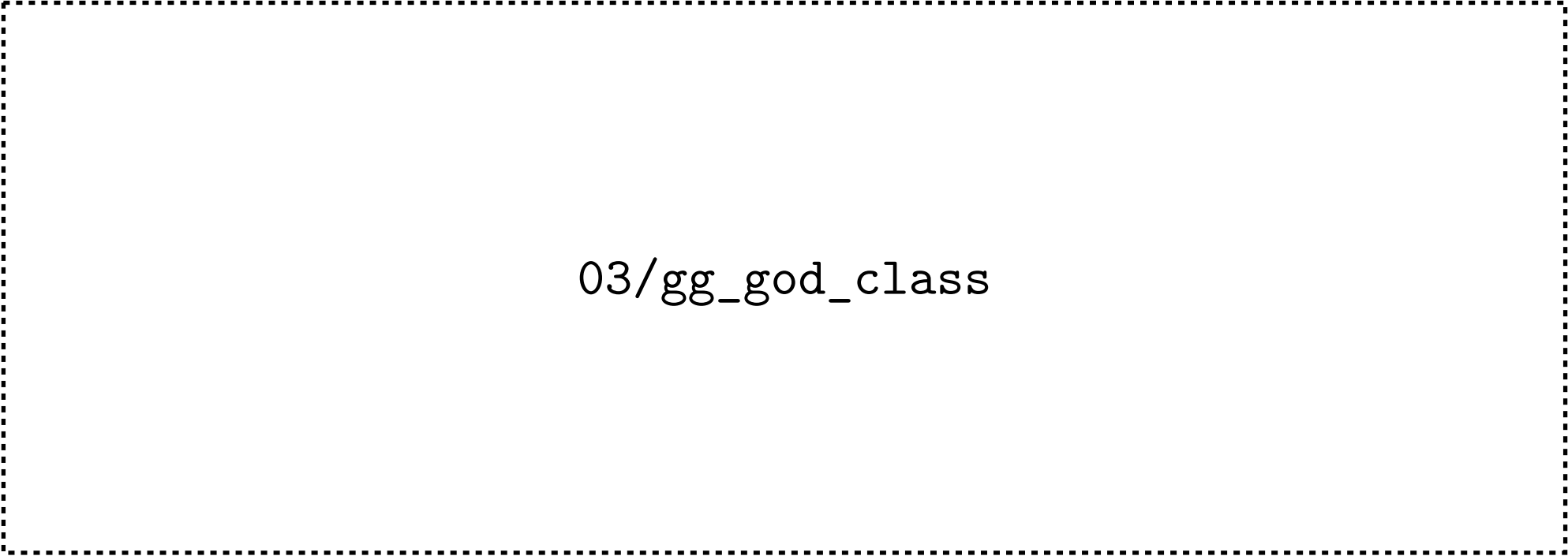
```
class A:  
    def __init__(self, ...):  
        self.b: B = ...  
  
    def f(self, c: C):  
        ...
```

- Here, A *depends* on B and C

Coupling and Dependencies

- **Coupling:** Degree of *dependency* between two objects
- A is *coupled* to B and C
 - ⇒ Changing B or C might mean A should change as well
 - ⇒ Using A in another codebase means B and C should be taken as well
- Decoupled code is *easier to change*

Modularization Using Classes



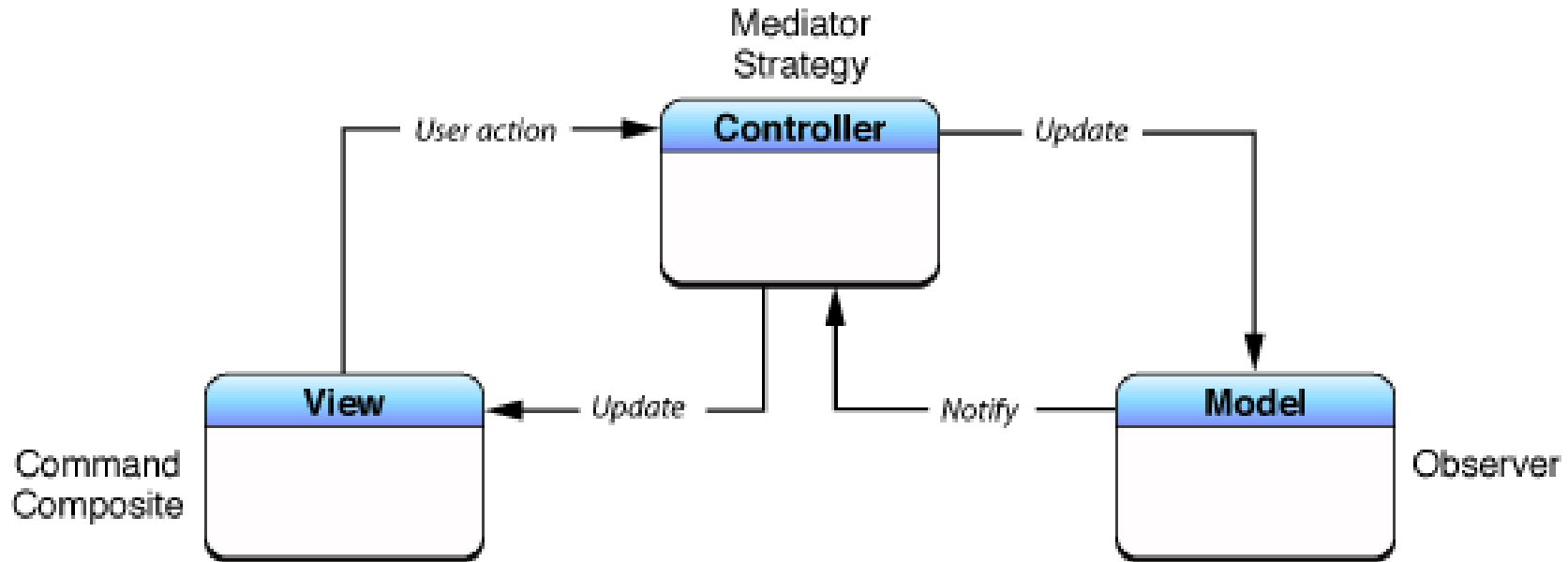
03/gg_god_class

How do we separate the underlying concerns?

Model-View-Controller (MVC)

- **Model-View-Controller:** Architectural pattern that organizes code into three nonoverlapping parts
- “MVC” has evolved to have several definitions; we will use this one:
 - **Model:** Encapsulates state and domain logic (independent of UI)
 - **View:** Displays interface and takes user input
 - **Controller:** “Glues” the Model and View

Model-View-Controller (MVC)



More accurate term: Model-View-Presenter (MVP) ([image source](#))

Model Design

- Design idea: Can play entire game “manually” by sending messages to Model
 - Correct order of messages must be sent
 - Define object as single point of communication with outside (e.g., `GuessingGameModel`)
 - Handles messages from outside the Model

Use Cases

- **Use case:** How actors can use the system to accomplish a certain goal
 - Software is created for different purposes
 - User interactions can be divided into use cases

Guessing Game MVC

- State needed for game:
 - Correct answer
 - Number of attempts left
- Use cases:
 - Start game
 - Make guess
 - Check whether guess is correct
 - Check whether game is done

Guessing Game MVC

- `GuessingGameModel`: Manages game state
- `GuessingGameController`: Has game loop; talks to both `Model` and `View`
- `GuessingGameView`: Manages terminal I/O
- `Model` and `View` are decoupled from each other
 - Can change `View` without changing `Model`
 - In general, we could decompose these objects further (if needed)

Demo: Guessing Game MVC

03/gg_mvc

Demo: Guessing Game Unit Tests

03/gg_unit_tests